

DDLRadar 概要设计文档

1. 引言

1.1 编写目的

本文档为 DDLRadar 系统概要设计文档，主要用于在需求规格说明书的基础上，进一步说明系统的总体架构、模块划分、数据结构、接口设计和核心业务流程，为后续编码实现、联调测试和项目维护提供依据。与需求文档关注“系统要做什么”不同，本文档着重回答“系统打算怎么实现”。

1.2 设计依据

《DDLRadar 软件需求规格说明书》

1.3 设计范围

涵盖系统的任务管理、风险等级计算、数据可视化看板（Dashboard）以及 AI 辅助模块的后端实现机制、数据库表结构及接口规范。

2. 系统总体设计

2.1 系统设计目标

实现一个轻量、易部署、稳定可演示的学业任务管理系统，重点保障数据增删改查的闭环，以及风险计算和 AI 建议两大核心亮点的无障碍演示。

2.2 系统总体架构

DDLRadar 采用前后端分离的设计方式。前端使用 HTML、CSS、JavaScript 实现页面展示和用户交互，后端使用 Rust + Axum 提供 REST API，数据库使用 SQLite 进行任务数据持久化存储。

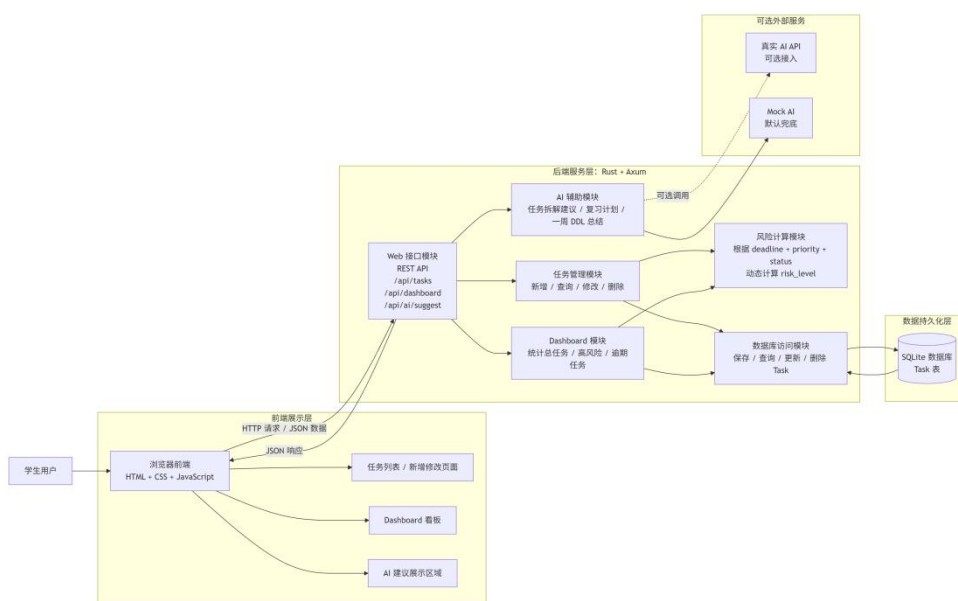


图 1 系统总体结构图

图 1 为系统总体结构图，通过该图，可以更直观地说明系统各组成部分之间的关系，即用户、前端页面、后端服务、业务模块与数据库之间的交互过程。该图能够体现系统采用前后端分离架构，前端负责用户输入与结果展示，后端负责业务处理与接口响应，数据库负责持久化保存任务数据。

2.3 技术架构说明

- 开发语言与框架：主线技术栈明确采用 Rust、Axum Web 框架。
- 数据流转与序列化：采用 serde / serde_json 进行前后端 JSON 数据的解析与处理。
- 日期处理：采用 chrono 库处理截止时间与系统时间的比对逻辑。
- AI 降级策略：AI 模块优先采用 Mock AI 模式保证稳定，并在接口设计上预留真实 AI API 的扩展空间。若后续接入真实 AI API，则需要在调用失败或超时时切换至 Mock AI。

2.4 系统运行环境

- 服务器端：支持安装了 Rust 工具链的本地操作系统。
- 客户端：主流的现代 Web 浏览器。

3. 系统模块设计

3.1 模块划分概述

系统按照功能职责划分为任务管理、风险计算、Dashboard、AI 辅助、数据库访问、Web 接口和前端展示等模块，各模块间通过接口和数据模型进行协作。

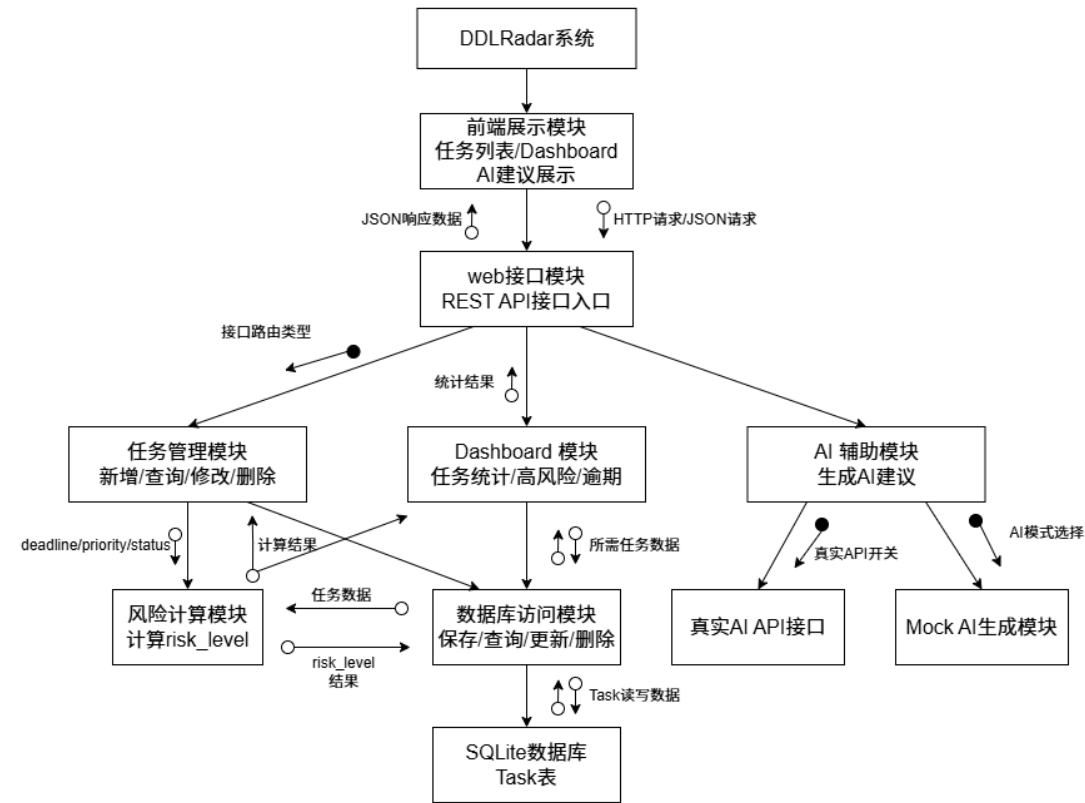


图 2 模块结构图

3.2 模块职责详述

通过明确各模块的边界,可以避免不同成员在开发过程中出现功能重复或职责不清的问题,也便于后续进行单元测试、接口联调和文档维护。

模块	主要职责
任务管理模块	负责任务项的增删改查
风险计算模块	根据 <code>deadline</code> , <code>priority</code> , <code>status</code> 计算风险等级
Dashboard 模块	汇总任务数量、高风险任务、逾期任务等统计数据
AI 辅助模块	根据任务类型生成任务拆解建议、复习建议或 DDL 总结
数据库模块	负责 Task 数据的保存、查询、更新和删除
Web 接口模块	对外提供 REST API
前端展示模块	展示任务列表、Dashboard、AI 建议等内容

表 1 模块职责详述表格

表 1 是对本系统主要模块及其职责的具体说明。其中,任务管理模块和数据库模块构成系统的基础数据层,负责保证任务数据能够被正确保存、查询和更新;风险计算模块和 Dashboard 模块构成系统的分析展示层,负责根据任务数据生成风险等级和统计结果;AI 辅助模块属于增强功能层,在不影响任务管理主流程的前提下,为用户提供任务拆解和复习建议。

4. 数据库概要设计

4.1 核心实体设计

基于开发时间的考虑,系统采用精简设计,因此核心实体即为 Task。由于本项目当前阶段不实现登录注册和多用户权限管理,因此数据库设计采用单核心实体方案。系统以 Task 任务实体作为数据管理中心,所有任务管理、风险计算、Dashboard 统计和 AI 建议功能均围绕 Task 数据展开。

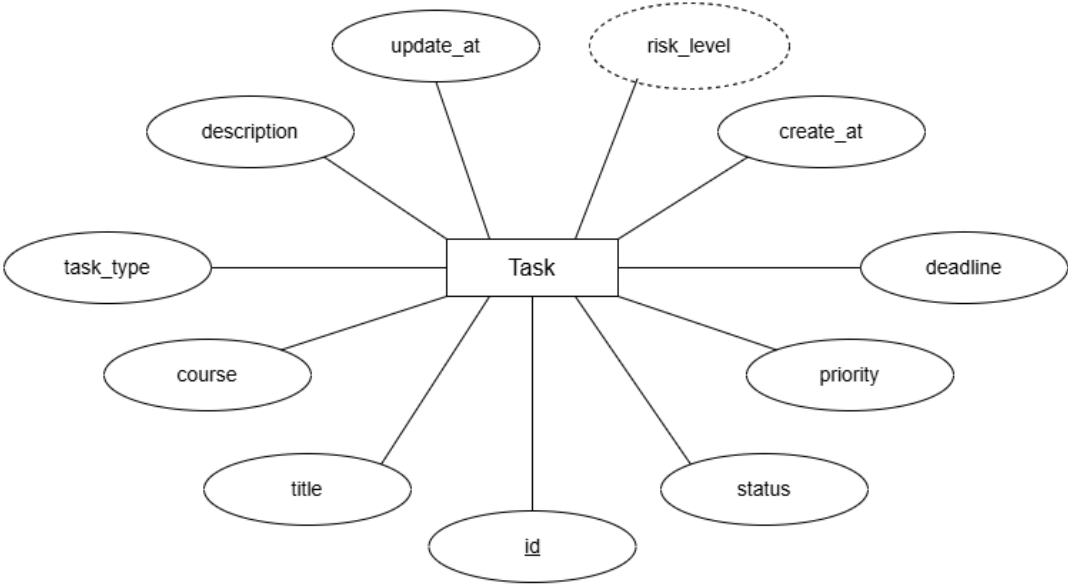


图 3 E-R 图

图 3 能够较为直观地展示系统的数据模型。值得注意的是,由于该系统只存在单实体,故不存在菱形。

4.2 Task 数据表设计

数据表的设计关乎整个系统前后端的统一与稳定,通过表格可以较好地展示不同字段的类型和功能。表 2 展示了 Task 数据表的主要字段设计。该表主要用于保存用户录入的原始任务信息,包括任务标题、所属课程、任务类型、截止时间、优先级、状态和任务描述等。

字段名	类型	说明
id	INTEGER	任务唯一标识(主键、自增)
title	TEXT	任务标题
course	TEXT	所属课程
task_type	TEXT	任务类型
deadline	TEXT	截止时间,采用 YYYY-MM-DD HH:MM 格式字符串
priority	TEXT	优先级
status	TEXT	当前状态
description	TEXT	任务描述
create_at	TEXT	任务创建时间
update_at	TEXT	任务更新时间

表 2 Task 数据设计表

需要说明的是,风险等级字段 risk_level 不作为数据库持久化字段保存,而是由风险计算模块根据 deadline、priority 和 status 在接口返回时动态计算生成。这样可以避免因时间变化导致数据库中风险等级与实际状态不一致。

4.3 枚举字段说明

- task_type: homework(作业)、exam(考试)、project(课程项目)、other(其他).
- priority: low(低)、mid(中)、high(高).
- status: todo(未开始)、doing(进行中)、done(已完成)
- risk_level: low(低风险)、mid(中风险)、high(高风险)、overdue(已逾期).

需要说明的是, status 表示用户主动维护的任务完成状态, risk_level 表示系统根据 deadline、priority 和 status 动态计算得到的风险状态。已逾期不作为用户手动填写的 status,而作为系统计算得到的 risk_level 展示。

4.4 数据存储说明

系统使用 SQLite 本地单文件数据库,数据文件默认存储在项目根目录(如 ddlradar.db),无需配置外部连接池,确保项目能够随时一键启动。

5. 接口概要设计

5.1 REST API 总体说明

所有接口均基于 HTTP 协议,返回 Content-Type 统一为 application/json。后端接口前缀统一设计为 /api/。

5.2 核心接口清单

接口路径和请求方法保持统一,有助于前后端分离开发和后续接口测试。表 3 展示了系统后端想向前端提供的核心 REST API。前端页面通过这些接口完成

任务数据的增删改查、Dashboard 数据获取以及 AI 建议请求。

接口路由	请求方法	功能说明
/api/health	GET	健康检查（验证后端存活状态）
/api/tasks	GET	获取任务列表
/api/tasks	POST	新增任务
/api/tasks/:id	PUT	修改任务
/api/tasks/:id	DELETE	删除任务
/api/dashboard	GET	获取 Dashboard 统计数据
/api/ai/suggest	POST	获取 AI 建议

表 3 核心接口表

5.3 接口返回格式约定

系统接口统一采用 JSON 格式返回数据。正常情况下，响应结果包含 success、message 和 data 字段。其中 success 表示请求是否成功，message 表示提示信息，data 用于承载实际返回数据。该设计便于前端统一处理接口成功和失败情况。

6. 核心业务流程设计

本章主要描述系统中几个关键业务流程的处理逻辑。与需求文档中描述“用户可以完成哪些操作”不同，本章重点说明系统在收到用户请求后，内部各模块如何协作完成数据校验、数据库访问、风险计算、统计生成和 AI 建议返回等处理过程。

6.1 新增任务流程

前端校验必填字段（标题、类型、截止时间等）-> 封装为 JSON 发起 POST 请求 -> 后端 Rust 反序列化并进行二次校验 -> 调用数据库访问模块，通过 rusqlite 写入 SQLite -> 成功后返回 201 Created 及任务对象。

6.2 查询任务流程

前端发起 GET 请求 -> 后端查询 SQLite 获取所有记录 -> 后端查询 SQLite 获取所有任务记录 -> 调用风险计算模块动态计算每个任务的 risk_level 和逾期状态 -> 随后按照 deadline 正序排列 -> 将包含风险等级的 JSON 列表返回给前端 -> 按照 deadline 正序排列 -> 返回 JSON 列表给前端。

6.3 风险等级计算流程

风险等级计算是 DDLRadar 系统的核心业务逻辑之一。系统不要求用户手动填写风险等级，而是根据任务截止时间、任务优先级和当前完成状态自动计算。该设计可以降低用户操作负担，并保证 Dashboard 中的高风险任务和逾期任务统计具有一致的数据来源。

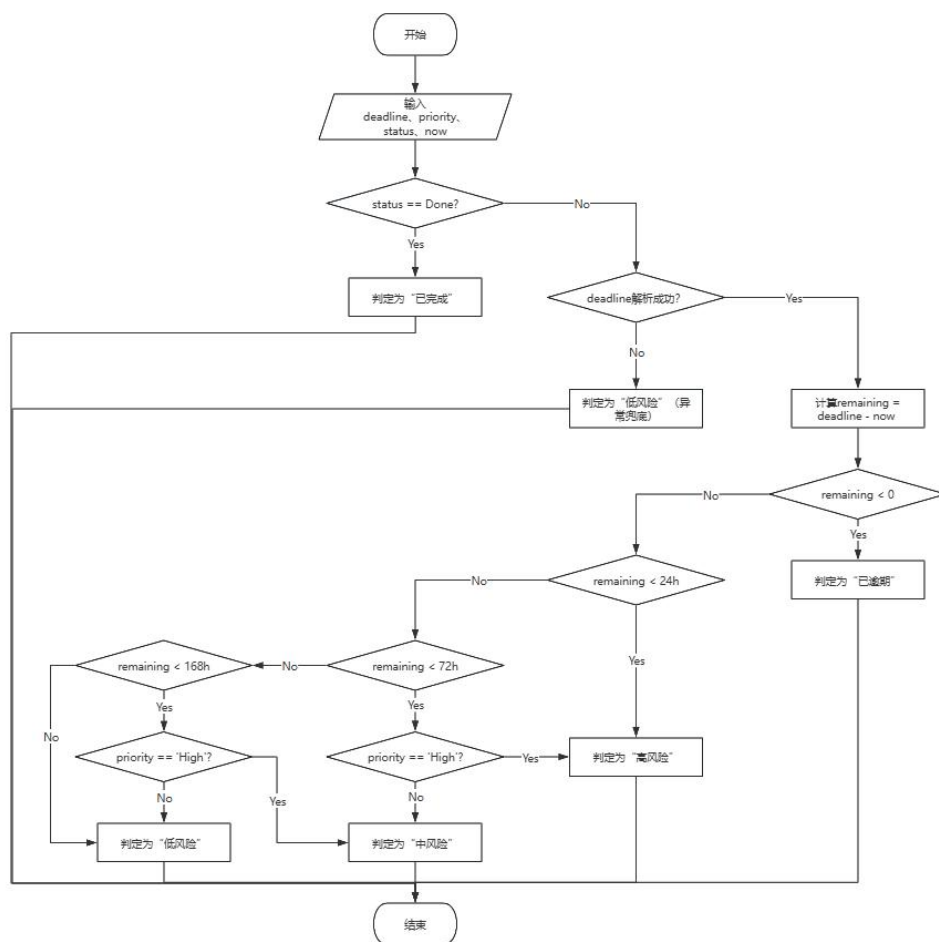


图 4 风险等级计算流程图

如图 4，在该流程中，系统首先判断任务是否已完成；若任务已完成，则直接返回“已完成”。若任务未完成，则继续判断当前时间是否超过 **deadline**，超过则返回“已逾期”。对于未逾期任务，系统根据剩余时间区间和优先级计算低风险、中风险或高风险。

6.4 AI 建议生成流程

AI 建议生成流程属于系统的增强功能，不影响任务 CRUD 和 Dashboard 的基本运行。系统根据任务类型选择不同的建议生成策略，例如课程项目类任务生成任务拆解建议，考试类任务生成复习计划建议，作业类任务生成完成步骤建议等。图 5 为 AI 建议生成流程图，能够展示用户使用 AI 生成建议的大致流程。

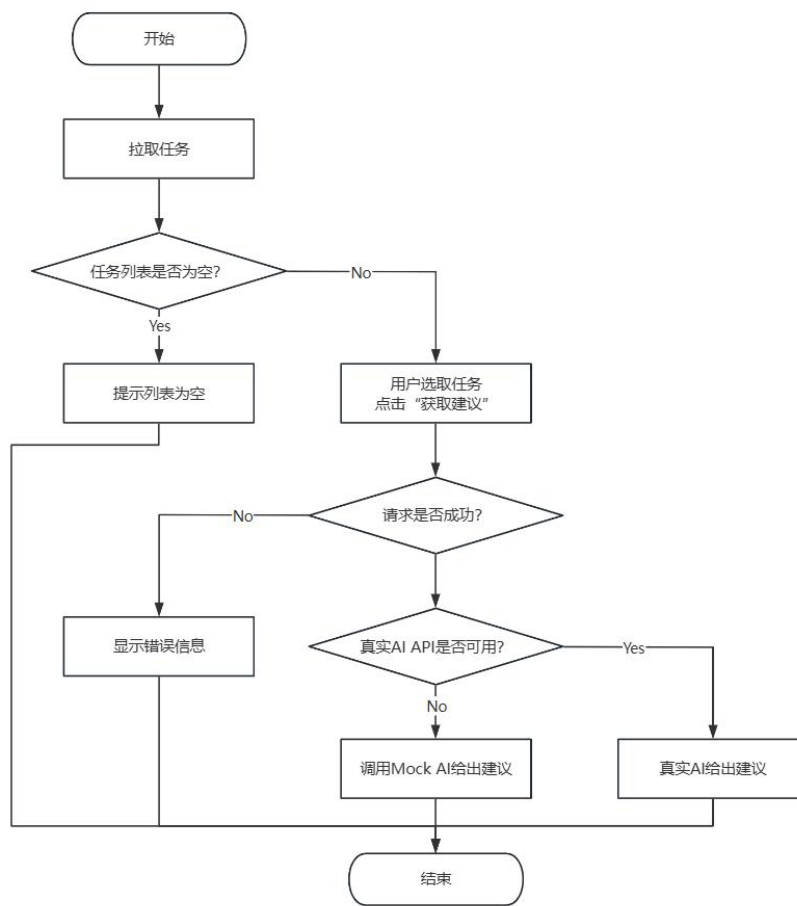


图 5 AI 建议生成流程图

为保证课程演示稳定性，AI 模块优先采用 Mock AI 模式返回模板化建议。若后续接入真实 AI API，则应保留超时处理和失败降级机制，避免外部接口异常导致系统核心功能不可用。

7. 异常处理与稳定性设计

7.1 日期格式异常处理设计

前端输入限制使用标准化控件；后端强制校验统一的日期时间字符串格式 YYYY-MM-DD HH:MM，解析失败时返回 HTTP 400 错误。

7.2 数据库操作异常处理

如遇主键冲突、锁表等数据库内部异常，后端捕获后返回 HTTP 500 内部服务器错误，并在日志中记录详细 trace。

7.3 AI 接口失败降级处理

接入真实 LLM API 时，使用 request 设置 5 秒超时，一旦触发 Error 或 Timeout，立即平滑切换到 Mock AI 服务。

7.4 前后端交互异常处理

前端全局封装 fetch，对非 2xx 响应码进行拦截，并向用户展示友好的

Toast 提示。

8. 部署与运行设计

8.1 后端运行方式

在含有 Cargo.toml 的目录下，执行 `cargo run` 拉起服务，默认监听 127.0.0.1:3000。

8.2 前端运行方式

可直接通过浏览器打开 index.html 运行，或通过 Live Server 等轻量级静态服务器挂载。

8.3 数据库文件说明

系统首次启动若检测不到 .db 后缀的文件，将自动执行初始化 SQL 语句建表。

8.4 测试与验证方式

核心计算逻辑（如时间差计算）使用 Rust 内置的 `#[test]` 编写单元测试，可通过 `cargo test` 等指令进行验证。

9. 系统维护设计

9.1 代码维护设计

为保证系统后续易于修改和扩展，项目代码按照模块职责进行组织。任务管理、数据库访问、风险计算、Dashboard 统计和 AI 辅助功能分别放在相对独立的模块中，避免所有业务逻辑集中在同一个文件中。后续新增功能时，应优先在对应模块中扩展，不直接修改无关模块，降低代码耦合度。

9.2 数据维护设计

系统采用 SQLite 数据库保存任务数据。由于当前阶段不实现多用户系统，数据库主要维护 Task 任务数据。后续如需增加用户系统，可在现有 Task 表基础上增加 User 表，并在 Task 表中增加 user_id 字段建立关联。当前版本中，风险等级和逾期状态不作为持久化字段保存，而是由系统根据 deadline、priority 和 status 动态计算，以避免时间变化导致存储数据失效。

9.3 接口维护设计

系统后端接口统一采用 /api/ 前缀，并使用 REST 风格组织任务管理、Dashboard 和 AI 建议相关接口。后续新增接口时，应保持统一的响应格式，包括 success、message 和 data 字段，以方便前端统一处理成功和失败情况。修改已有接口时，应同步更新接口文档，避免前后端联调时出现字段不一致问题。

9.4 AI 模块维护设计

AI 辅助模块采用可替换的设计思路。当前阶段优先使用 Mock AI 模式生

成模板化建议，以保证系统在无网络或无 API Key 的情况下仍能稳定演示。若后续接入真实 AI API，应保留 Mock AI 作为兜底方案，并对真实 API 调用设置超时时间和异常处理，防止外部服务异常影响系统核心功能。

9.5 文档维护设计

项目开发过程中应持续维护需求文档、概要设计文档、接口说明、测试报告和 AI 使用说明。当系统功能、接口字段、数据库结构或风险计算规则发生变化时，应及时同步更新相关文档。最终提交前，应确保文档内容与实际代码实现保持一致。

10. 小结

本概要设计文档界定了 DDLRadar 项目的后端架构、数据模型(Task 实体)、REST API 接口规范以及最核心的风险定级与 AI 降级业务流程，为下一步的详细编码和数据库创建铺平了道路。